Spatial model fitting walkthrough

Chad Babcock

1 Spatial model definition

We can define a spatial regression model as

$$y(\boldsymbol{s}) \sim \mathcal{N}(\mu(\boldsymbol{s}) + w(\boldsymbol{s}), \tau^2), \tag{1}$$

where $\mu(\boldsymbol{s}) = \boldsymbol{x}^T(\boldsymbol{s})\boldsymbol{\beta}$ and models the relationship between the covariates, $\boldsymbol{x}(\boldsymbol{s})$, and the response, $y(\boldsymbol{s})$. $\boldsymbol{\beta}$ is a vector of regression coefficients to be estimated. $\boldsymbol{s}$ is a two-dimensional coordinate vector defining a point location in the spatial domain. τ^2 is the non-spatial variance. In kriging speak, this is the *nugget*. The spatial random effect, $w(\boldsymbol{s})$, is modeled as a Gaussian process ($\mathcal{GP}$) like so,

$$w(\boldsymbol{s}) \sim \mathcal{GP}(0, \sigma^2\rho(\boldsymbol{s}, \boldsymbol{s}^*; \phi)), \tag{2}$$

where σ^2 is the spatial variance. In kriging terms, this would be called the *partial sill*. It describes all the variability in $\boldsymbol{y}$ (after accounting for $\boldsymbol{\mu}$) that is spatially structured. The larger σ^2 is compared to τ^2, the more spatially structured variability there is. $\rho(\boldsymbol{s}, \boldsymbol{s}^*; \phi)$ is an autocorrelation function. There are a number of autocorrelation functions one can use, but I pretty much exclusively use an exponential correlation function. So, $\rho(\boldsymbol{s}, \boldsymbol{s}^*; \phi) = \exp(-\phi||\boldsymbol{s} - \boldsymbol{s}^*||)$, where $||\boldsymbol{s} - \boldsymbol{s}^*||$ is the Euclidean distance between locations $\boldsymbol{s}$ and $\boldsymbol{s}^*$. ϕ is the correlation decay parameter and is related to the *spatial range*. Notice that, as the separation distance between $\boldsymbol{s}$ and $\boldsymbol{s}^*$ becomes longer, the correlation between $\boldsymbol{s}$ and $\boldsymbol{s}^*$ lowers. ϕ controls how fast the correlation drops as separation distance increases. The larger the ϕ, the faster the correlation drops, i.e., the shorter the *spatial range*.

There are several parameters in model (1) that we would like to estimate. We want to estimate $\boldsymbol{\beta}$, $\boldsymbol{w}$, τ^2, σ^2 and ϕ. In Section 3, I walk through how to use `spBayes` to get samples from the posterior distribution of the parameters in model (1). In Section 2, I walk through how to generate a synthetic dataset that exhibits spatially structured variability. The dataset we generate in Section 2 is the one we will use to fit the spatial model in Section 3.

In Section 4, I show how to use `spBayes` to predict y at unobserved locations. I also show how to calculate small area estimates in Section 4. In Section 5, I conduct a resampling experiment to compare uncertainty interval coverage probabilities for the spatial mean of $\boldsymbol{y}$ generated using model (1) and a traditional design-based approach.

2 Synthetic population

Here we will look at the code used to generate the synthetic population. This code is contained in the R script `gen-synthetic-population.R`. First, we load up the libraries we need.

```
> ###############
> ## libraries ##
> library(sp)
> library(spBayes)
> library(MASS)
```

Then we create a grid of coordinates. We can think of these as grid cell centers. `coords` holds the coordinates for the grid cell centers in an $N \times 2$ matrix where N is the number of grid cells. We will also set up a matrix holding the intersite distances between all the grid cell centers. `dmat` is an $N \times N$ matrix where $\texttt{dmat}_{ij}$ holds the distance between grid cell i and grid cell j.

```
> coords <- as.matrix(expand.grid(seq(0,1,length.out = 40),seq(0,1,length.out = 40)))
> N <- nrow(coords)

> dmat <- iDist(coords)
```

Now we will simulate a covariate vector ($\boldsymbol{x}$). And we are going to give it some spatial structure. `x.sigma.sq` is the spatial variance for $\boldsymbol{x}$. `x.phi` is the spatial decay parameter. `x.mu` is the mean vector. `x.cov` is the covariance matrix for $\boldsymbol{x}$. Notice that `x.cov` is generated by multiplying σ^2 by an exponential correlation function. Finally, we draw one realization from a multivatiate random normal with our specified mean vector and covariance matrix. This one realized surface is now the covariate in our synthetic population.

```
> x.sigma.sq <- 2
> x.phi <- 3/.15
> x.mu <- rep(5,N)

> x.cov <- x.sigma.sq*exp(-x.phi*dmat)

> x <- mvrnorm(1,x.mu,x.cov)
```

We use the same approach we used to get $\boldsymbol{x}$ to simulate $\boldsymbol{w}$.

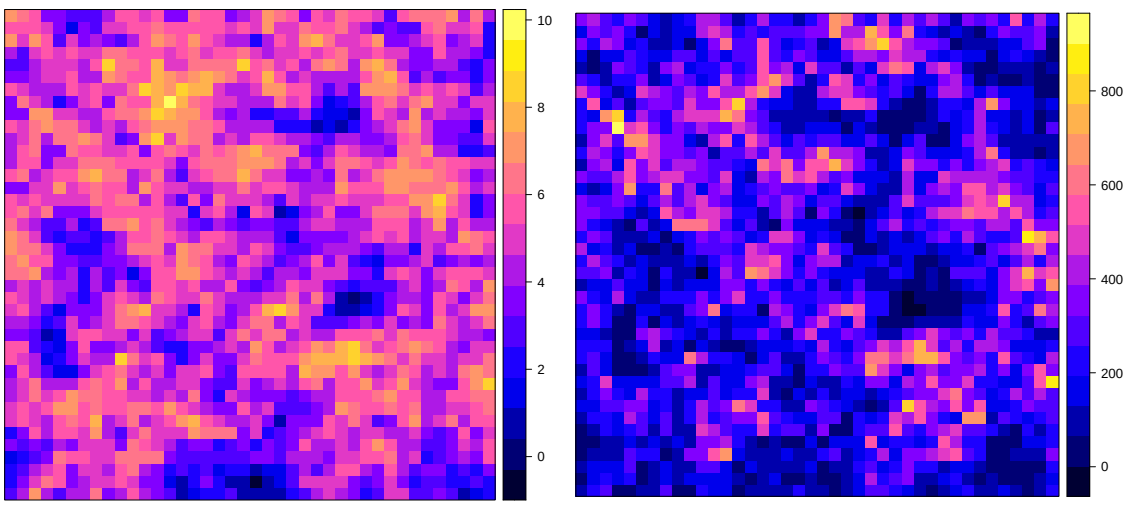
```
> w.sigma.sq <- 7
> w.phi <- 3/.25

> w.cov <- w.sigma.sq*exp(-w.phi*dmat)

> w <- mvrnorm(1,rep(0,N),w.cov)
```

Now that we have $\boldsymbol{x}$ and $\boldsymbol{w}$, we can simulate $\boldsymbol{y}$. First we set our β parameters and τ^2. Then we draw our $\boldsymbol{y}$ realization from a normal with a mean equal to our linear model plus the spatial random effect, $\boldsymbol{w}$ ($\beta_0 + \beta_1 \boldsymbol{x}_1 + \boldsymbol{w}$). I square the normal draws here and when we fit the model later, we will use a squareroot transformation. Typically, when we try to model forest characteristics like basal area, tree density or volume, we want to make sure our model has only positive support (does not predict negative numbers). Using a squareroot transformation is one way to accomplish this.

```
> beta.0 <- 1
> beta.1 <- 3
> tau.sq <- 2

> y.mu <- beta.0 + x*beta.1 + w

> y <- rnorm(N,y.mu, sqrt(tau.sq))^2
```

Then we save $\boldsymbol{x}$, $\boldsymbol{y}$ and `coords` as an RData file.

```
> save(x,y,coords, file = "synthetic-data.RData")
```



(a) A map of $\boldsymbol{x}$

(b) A map of $\boldsymbol{y}$

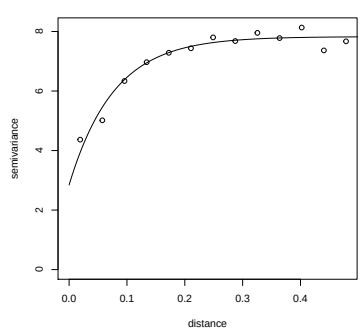
3 Model fitting in spBayes

Here we will go through the contents of `model-fitting-example.R` to show how to estimate model (1) using the synthetic dataset simulated in Section 2. First we load up libraries and the synthetic dataset we saved from Section 2. We are going to select 250 grid cells to be our *plots* where both x and y were measured. We will also set the number of MCMC samples (`n.samples`) we want and the number of β parameters we have in our model (`p`).

```
> ###############
> ## libraries ##
> library(sp)
> library(spBayes)
> library(geoR)

> load("synthetic-data.RData")

> n <- 250 ## number of plots we want to imagine we collected
> sub <- sample(1:nrow(coords), n)
> y <- y[sub]
> x <- x[sub]
> coords <- coords[sub,]

> n.samples <- 5000

> p <- 2
```

Now we will fit a model with no spatial random effect using `lm` and use the residuals to make a semi-variogram and semi-variogram model. We can use the semi-variogram to inform our prior selection.

```
> m.0 <- lm(sqrt(y) ~ x)
> vario <- variog(data = resid(m.0), coords = coords, max.dist = .75)
> vario.mod <- variofit(vario, cov.model = "exponential")
```

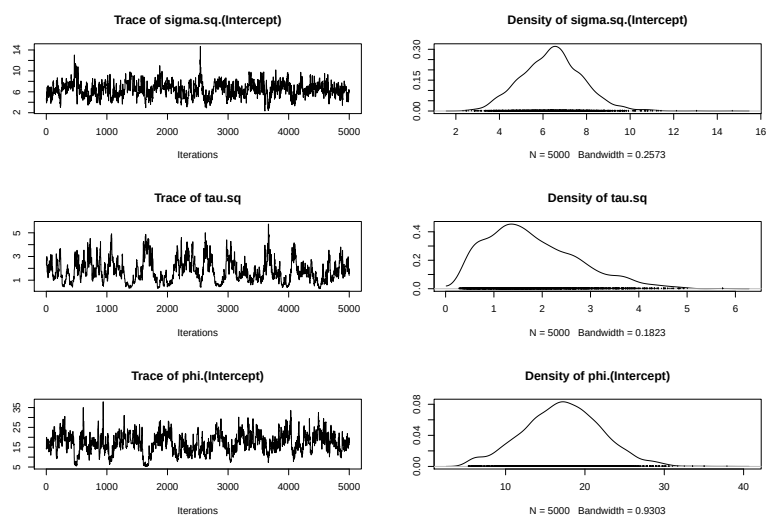


Now we define prior distributions for $\boldsymbol{\beta}$, ϕ, σ^2 and τ^2. We set the $\boldsymbol{\beta}$ prior to a normal distribution with mean $\mathbf{0}$ and a covariance matrix with large diagonal elements (1000) and all off-diagonal elements set to zero. This is a vague prior. The larger the diagonal elements, the more vague the prior. ϕ is set to a uniform prior with positive support. In an ideal world, we would like to set the bounds of the uniform prior to include all possible *spatial ranges* between zero and the maximum separation distance between locations within the domain. In practice though, ϕ can be incredibly hard to estimate. Constraining the ϕ prior can help to estimate it. σ^2 and τ^2 both receive inverse gamma priors. Setting the a inverse gamma parameter to 2, gives the prior a mean equal to the b parameter and an infinite variance. Then we can set the b parameter for σ^2 to what looks like the *partial sill* from the semi-variogram. The b parameter for τ^2 can be set to what we think the *nugget* is based on the semi-variogram.

```
> priors <- list("beta.Norm"=list(rep(0,p), diag(1000,p)),
>                "phi.Unif"=list(3/1, 3/.001),
>                "sigma.sq.IG"=list(2, 6),
>                "tau.sq.IG"=c(2, 2))
```

Now we set starting and tuning values for ϕ, σ^2 and τ^2. Then we can run the model.

```
> starting <- list("phi"=3/.25, "sigma.sq"=6, "tau.sq"=2)

> tuning <- list("phi"=0.04, "sigma.sq"=0.04, "tau.sq"=0.04)

> m.1 <- spSVC(sqrt(y) ~ x, coords = coords, starting = starting, tuning = tuning,
>             priors = priors, cov.model = "exponential", n.samples = n.samples)

> plot(m.1$p.theta.samples)
> save(m.1, file = "fitted-model.RData")
```



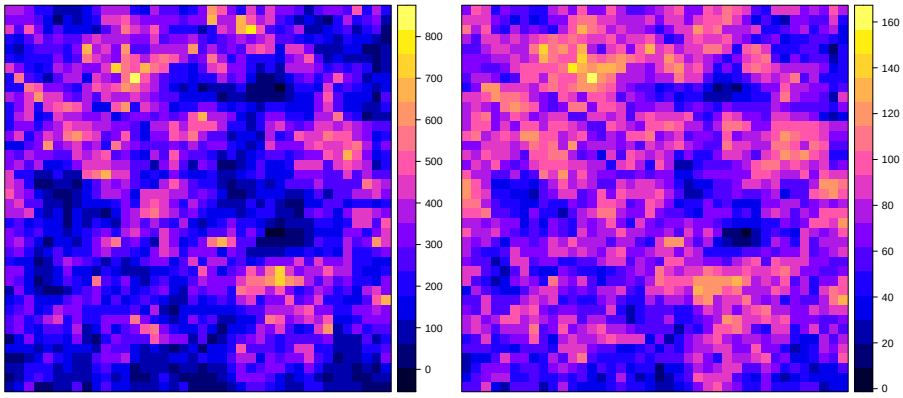
4 Spatial prediction and areal estimation

Now we can look at the `prediction-example.R` code. First we load up libraries, the synthetic dataset and the fitted model from Section 3.

```
> ###############
> ## libraries ##
> library(sp)
> library(spBayes)
> library(MASS)

> ## load up synthetic data
> load("synthetic-data.RData")

> ## load up fitted model
> load("fitted-model.RData")
```

We are going to predict two ways here. The first will be using the `spPredict` function in the `spBayes` package with the `joint` argument set to `FALSE` (default). The next block of code samples from the posterior predictive distribution (ppd) for $\boldsymbol{y}$ at all 1600 grid cells. Then we back-transform the posterior predictive samples by squaring them. Then we use the quantile function to calculate posterior medians and 95% credible bounds. We also get the standard deviation of the ppd at each grid cell which is nice for mapping uncertainty.

```
## First we recover beta samples and w samples. We also burn in and thin in this step.
> m.1 <- spRecover(m.1, start = 501, thin = 4)

## spPredict doesn't allow us to predict exactly at observation locations.
## So we are going to do a little hack to trick the function into working.

## We are going to move our observation locations stored in the model object a small amount.
## We would not have to do this if we didn't need to predict at observation locations.
> m.1$coords[,1]  <-  m.1$coords[,1] + .000000000001

> m.p <- spPredict(m.1, pred.coords = coords, pred.covars = cbind(1,x))

> y.ppd <- m.p$p.y.predictive.samples^2 ## backtransform the samples

> y.ppd.cred <- t(apply(y.ppd, 1, quantile, probs = c(.5,.025,.975)))
> y.ppd.sd <- apply(y.ppd,1,sd)
```



(a) Predictions of $\boldsymbol{y}$ (median of ppd)

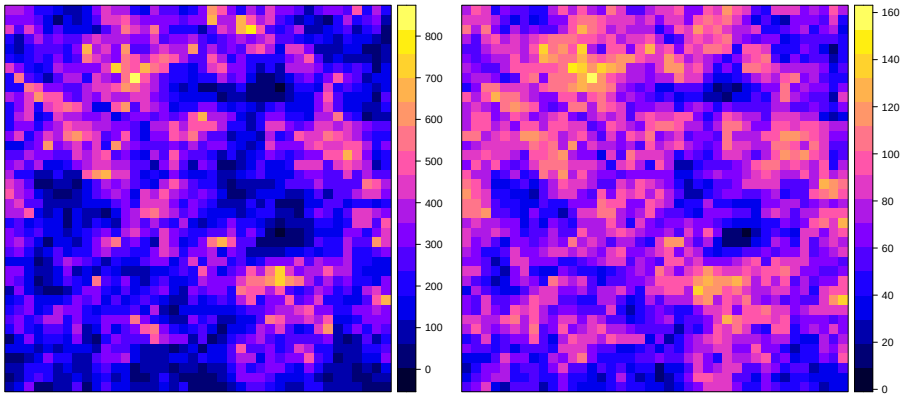
(b) Uncertainty for $\boldsymbol{y}$ (standard deviation of ppd).

Running `spPredict` with `joint = FALSE` is great for mapping and getting grid cell level uncertainty estimates. However, when joint = FALSE, `spPredict` samples each grid cell independently rather than sampling all grid cells jointly. This means that posterior predictive samples from spPredict with `joint = FALSE`, cannot be used to get areal estimates (i.e., no small area or large area estimation). Let's try setting `joint = TRUE`.

```
## running spPredict with joint=TRUE will take longer to run.
## It's not a huge difference here, but for larger datasets, it can be significant.
> m.p.joint <- spPredict(m.1, pred.coords = coords, pred.covars = cbind(1,x), joint = TRUE)

> y.joint.ppd <- m.p.joint$p.y.predictive.samples^2 ## backtransform the samples

> y.joint.ppd.cred <- t(apply(y.joint.ppd, 1, quantile, probs = c(.5,.025,.975)))
> y.joint.ppd.sd <- apply(y.joint.ppd,1,sd)
```

Now that we have samples from the joint posterior predictive distribution, we can make hay in terms of areal estimation. To get the posterior for the mean of $\boldsymbol{y}$ for the entire area we take the column means of `y.joint.ppd`.

```
> y.mn.ppd <- colMeans(y.joint.ppd)
> quantile(y.mn.ppd, probs = c(.5,.025,.975))
     50%     2.5%    97.5%
263.2876 253.3897 273.6944

> sd(y.mn.ppd)
[1] 5.160775
```



(a) Predictions of $\boldsymbol{y}$ (median of joint ppd) (b) Uncertainty for $\boldsymbol{y}$ (standard deviation of joint ppd).

See what happens when we use `spPredict` with `joint=FALSE` though.

```
> y.mn.ppd2 <- colMeans(y.ppd)
> quantile(y.mn.ppd2, probs = c(.5,.025,.975))
     50%     2.5%    97.5%
263.4321 256.1389 271.0192

> sd(y.mn.ppd2)
[1] 3.829458
```

It estimates a much lower uncertainty which is wrong.

We can use the joint ppd for any area within the full domain too. The first 800 grid cells from our synthetic dataset make up the bottom half of the domain. To get the ppd for the bottom half of the domain all we have to do is use the joint ppd in that area.

```
> y.bottom.mn.ppd <- colMeans(y.joint.ppd[1:800,])
```

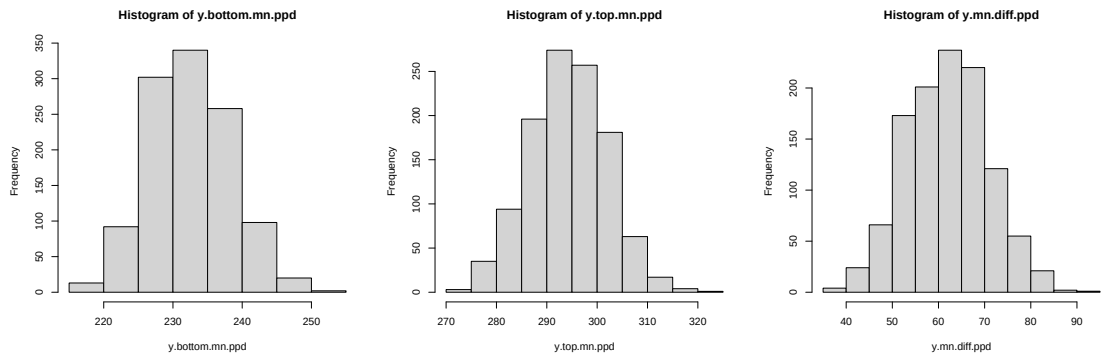
And we can do the same for the top half.

```
> y.top.mn.ppd <- colMeans(y.joint.ppd[801:1600,])
```

Looks like the mean of $\boldsymbol{y}$ for the top half is larger than the mean of $\boldsymbol{y}$ for the bottom half. We can conduct a statistical test for this even.

```
> y.mn.diff.ppd <- y.top.mn.ppd-y.bottom.mn.ppd
> quantile(y.mn.diff.ppd, probs = c(.5,.025,.975))
     50%     2.5%    97.5%
61.91653 45.00437 79.31892
```

Since the 95% credible interval does not include zero, we can be 95% sure that the mean of $\boldsymbol{y}$ from the top half is larger than the mean of $\boldsymbol{y}$ for the bottom half.



(a) ppd for bottom half of domain (b) ppd for top half of domain (c) ppd of top half minus bottom half

5 Resampling experiment

My goal here is to assess the coverage probability of the 95% credible intervals for the areal mean of $\boldsymbol{y}$ generated using `spPredict` with `joint=TRUE`. I'm testing two spatial models; one without using the covariate (*null spatial model*) and one using the covariate (*spatial model*). I'm also going to compare to design-based estimates. Results are based on models fit to 1000 simple random samples of size $n = 250$. Variance estimates for the spatial models were derived by taking the variance of the ppd for the mean of $\boldsymbol{y}$. 95% credible lower and upper bounds were calculated as the 2.5% and 97.5% quantiles of the ppd for the mean of $\boldsymbol{y}$, respectively.

	mean of estimated means	variance of estimated means	mean of variance estimates	95% CP
design-based	255.86	85.35	84.97	.943
null spatial model (`joint=F`)	256.08	62.09	23.33	.767
null spatial model (`joint=T`)	256.09	62.16	62.41	.947
spatial model (`joint=F`)	256.38	21.27	12.29	.865
spatial model (`joint=T`)	256.38	21.27	22.73	.950

We see that both the null and full spatial model credible intervals have good empirical 95% coverage when `joint = TRUE`. The coverage is no good when `joint = FALSE`. The average of estimated variances for the full model is substantially lower than the null model, indicating that the covariate helped us reduce uncertainty.